BASE24®

Files and Record Structures Reference Manual

06/2002, Edition 2, Release 6.0 Version 2

Contents

Section 2

Section 3

Appendix A

ACI Worldwide Inc.

Preface

The ***BASE24 Files and Record Structures Reference Manual*** provides general information about the structure of BASE24 files, records, and tables. It includes details on the location of the compiled Data Definition Language (DDL) dictionary for each file or record structure, the location of source schema files for each SQL table, and provides instructions on how to use the DDLDOC utility.

Audience

This manual is intended for use by technical personnel (e.g., system managers, network operators, and programmers) responsible for maintaining the BASE24 system after installation. Essentially, it guides the user in accessing DDL information online. The DDLs can be used to gain a better understanding of the files, tables, and records in the system or as a point of reference for other BASE24 publications that cite DDL field or data names in their descriptions. To effectively use the information in this manual, a reader should have an understanding of DDLs, files, tables, and record structures and how they relate to BASE24 processing.

Additional Documentation

The BASE24 documentation set is arranged so that each BASE24 manual presents a topic or group of related topics in detail. When one BASE24 manual presents a topic that has already been covered in detail in another BASE24 manual, the topic is summarized and the reader is directed to the other manual for additional information. Information has been arranged in this manner to be more efficient for readers that do not need the additional detail and at the same time provide the source for readers that require the additional information.

This manual contains references to the following BASE24 publications:

- The ***BASE24 Base Files Maintenance Manual*** and the various product-specific files maintenance manuals document BASE24 file maintenance screen fields.
- The ***BASE24 BIC ISO Standards Manual*** documents the ISO external message format as used by the BIC ISO Interchange Interface process.
- The ***BASE24 Classic 6.0 Installation Guide*** contains detailed information on modifying the Product Indicator Table File (PITABLE).
- The ***BASE24 CRT Access Manual*** documents the Security File (SEC), Network Control Supervisor Security File (NCSS), and Network Control Supervisor Profile File (NCSP) file maintenance screen fields.
- The ***BASE24 External Message Manual*** documents the BASE24-atm, BASE24-pos, BASE24-teller, and BASE24-from host maintenance ISO external message formats used by the ISO Host Interface and From Host Maintenance processes.
- The ***BASE24 ITS Kernel Configuration Manual*** documents the Extended Memory Table Build utility used for ITS-based applications.
- The ***BASE24 ITS Kernel Files Maintenance Manual*** describes the screens used to warmboot extended memory tables in an ITS environment.
- The ***BASE24 Refresh and Extract Operators Manual*** documents Refresh and Extract tape headers, tape trailers, and organizational record formats.
- The ***BASE24 Tokens Manual*** documents BASE24 tokens.
- The ***BASE24-atm Transaction Processing Manual*** documents the BASE24-atm Standard Internal Message (STM) and the Base Extended Memory Table Build utility.
- The ***BASE24-pos Transaction Processing Manual*** documents the BASE24-pos Standard Internal Message (PSTM) and the Base Extended Memory Table Build utility.
- The ***BASE24-teller Transaction Processing Manual*** documents the BASE24-teller Standard Internal Message (TSTM).
- The ***BASE24 Remote Banking Transaction Processing Manual*** documents the BASE24-telebanking Standard Internal Message (BSTM).
- The ***BASE24-pos Stored Value Support Manual*** documents the BASE24-pos Stored Value add-on product.

Software

This manual documents standard processing as of its publication date. Software that is not current and custom software modifications (CSMs) may result in processing that differs from the material presented in this manual. The customer is responsible for identifying and noting these changes.

Manual Summary

The following is a summary of the contents of this manual.

"Conventions Used in This Manual" follows this preface and describes notation and documentation conventions necessary to understand the information in the manual.

Section 1, "Introduction," provides an introduction to BASE24 file and table types and their usage.

Section 2, "The DDLDOC Utility," documents the DDLDOC utility, which allows you to print DDL information from any compiled Compaq NonStop data dictionary.

Section 3, "File, Table, and Record Structure Locations," identifies the standard online location for BASE24 file, table, and record structure information.

Appendix A, "ACI-Supplied DDLDOC Input Files," provides an overview of the input files provided by ACI for use with the DDLDOC utility.

Publication Identification

Three entries appearing at the bottom of each page uniquely identify this BASE24 publication. The publication number (for example, BA-AE000-11 for the ***BASE24 Files and Record Structures Manual***) appears on every page to assist readers in identifying the manual from which a page of information was printed. The publication date (for example, 06/2002 for June, 2002) and edition (for example, Ed. 2 for edition 2) indicate the issue of the manual. The software release information (for example, R6.0v2 for release 6.0, version 2) specifies the software that the manual describes. This information matches the document information on the copyright page of the manual.

ACI Worldwide Inc.

Conventions Used in This Manual

This section explains how command syntax notation is documented in this manual.

Syntax Notation

Syntax descriptions in this manual use several symbols and conventions to denote how commands must be entered. These conventions are described in the table below.

Notation	Meaning
UPPERCASE LETTERS	Indicate key words and literals that you must enter exactly as shown.
lowercase italics	Identify variables that you must provide.
Brackets [ ]	Enclose optional syntax items that you can enter in the command.
Braces { }	Enclose a group of items from which you are required to choose one item. The items are separated within the braces by a vertical bar (|).
Vertical bar |	Separates alternatives in a list of items.
Ellipsis …	Indicates the immediately preceding syntactical item can occur one or more times.
Punctuation	Must be entered as shown. This includes parentheses, commas, semicolons, and other symbols not previously described.
Spaces	Are required as delimiters wherever it would be otherwise impossible to discriminate between adjacent words or symbols. You can insert additional spaces between any adjacent words or symbols, but not within a word or multiple character symbol.

Notation	Meaning
Indented lines	Indicates a continuation of the preceding line of syntax. If the syntax of a command is too long to fit on a single line, each continuation line is indented.

Section 1

Introduction

This section contains general information on BASE24 files and tables. It includes information on the following topics:

- An explanation of BASE24 file and table types
- An explanation of the BASE24 segmented file structure
- An explanation of template files and how they are used by BASE24
- An explanation of how BASE24 files, tables, and record structures are documented

BASE24 Files and Tables

BASE24 uses a number of Enscribe files and SQL tables in its processing. For the purpose of discussion, these files and tables are divided into two categories: shared files and tables, and product files and tables. For performance reasons, some file and SQL table data can be accessed from extended memory tables held in processor memory rather than from files or tables on disk.

Shared Files and Tables

Shared files and tables are used by one or more BASE24 products. Examples of shared files are the Cardholder Authorization File (CAF), the Institution Definition File (IDF), and the Online Maintenance File (OMF). Examples of shared tables are the Customer Table (CSTT) and the Personal Information Table (PIT). These files and tables are used at the logical network level; in other words, each logical network has its own set of shared files and tables.

Product Files and Tables

Product files and tables are used by one BASE24 product. Examples of product files are the Teller Terminal Data File (TTDF), which is a BASE24-teller file, or the POS Retailer Definition File (PRDF), which is a BASE24-pos file. Examples of product tables are the Bank Table (BANK), and the Customer Vendor Table (CVND), which are BASE24-billpay tables. Like the shared files, each logical network has its own set of product files and tables.

Extended Memory Tables

For performance reasons, components of the BASE24 environment access critical Enscribe file or SQL table data from extended memory tables held in processor memory rather than from files or tables on disk. This saves time in the execution of time-critical operations. Some extended memory tables are shared by more than one process, module, or object while others are used by a single process, module, or object only. The Extended Memory Table Build utility provided with the BASE24 environment is used to build shared extended memory tables from disk files or tables. Extended memory tables that are not shared are built by the individual processes, modules, or objects that use them when they are initialized. Once an extended memory table has been built or rebuilt, it must be allocated or reallocated (i.e., warmbooted) to the processes, modules, or objects that use it before these processes, modules, or objects begin using it in processing.

For the BASE24-atm and BASE24-pos products, shared extended memory tables can be warmbooted using the EMT Control Commands screen (accessed as screen 5 from the DCT Menu) or by entering ALTER ATTRIBUTE text commands from a network control facility. For more information on the Extended Memory Table Build utility used to build extended memory tables used by the BASE24-atm and BASE24-pos products, refer to the ***BASE24-atm Transaction Processing Manual*** and the ***BASE24-pos Transaction Processing Manual***, respectively. These manuals also provide information on warmbooting these extended memory tables.

For ITS-based applications, the EMT WARMBOOT or LCONF WARMBOOT commands are used to warmboot extended memory tables. For more information on the Extended Memory Table Build utility used to build extended memory tables used by ITS-based processes, refer to the ***BASE24 ITS Kernel Configuration Manual***. For more information on the EMT WARMBOOT and LCONF WARMBOOT commands, refer to the ***BASE24 ITS Kernel Files Maintenance Manual***.

Segmented File Structures

For the integration of multiple BASE24 products, and to conserve disk space, many shared files have a segmented structure. Files with a segmented structure have records that are segmented according to the products that use them.

In an unsegmented file, all of the fields in the file are always present, whether the fields are used or not. If many of the fields in the record are not used because they relate to an unused feature, the result is wasted disk space.

File segmentation divides a file record into a series of smaller parts. Each part, or segment, consists of fields related to each other by product use or by feature. For example, the BASE24-atm segment of a file contains fields used solely by the BASE24-atm product, and the Preauthorized Holds segment contains fields related to the preauthorized holds function. When a record is created, it contains only those segments that are being used.

File segmentation allows the fields that are used exclusively by a single BASE24 product or function to become part of the file records only if the product or function is used. If the BASE24 product or function is not used, the segment containing the fields used by that product or function is not included in the file records. Depending on the type of file, the segments that can appear in the file are determined by the products supported at the financial institution or logical network level.

Financial Institution Level Authorization Files

Financial institution level authorization files contain customer account information needed to authorize a financial transaction. The following files are financial institution authorization files:

- Cardholder Authorization File (CAF)
- Negative Card File (NEG)
- Positive Balance File (PBF)
- Usage Accumulation File (UAF)

Separate authorization files can be specified for each financial institution in the Institution Definition File (IDF). Therefore, the segments that are included in these files depend on the products supported by the financial institution. The FIID AUTH FILE SEGMENT INDICATORS fields on IDF screens 5 and 6 specify the segments that are included in the authorization files.

Logical Network Level Configuration Files

Logical network level configuration files contain parameters that control the operation of the logical network. The following logical network configuration files are segmented:

- Card Prefix File (CPF)
- Enhanced Interchange Configuration File (ICFE)
- Extract Configuration File (ECF)
- Host Configuration File (HCF)
- Institution Definition File (IDF)
- Interchange Configuration File (ICF)

These files are named at the logical network level in the Logical Network Configuration File (LCONF). Therefore, the segments that are included in these files depend on the product segments supported by the logical network. The Segment Table in the Product Indicator Table File (PITABLE) indicates the segments that are included in these files. The PITABLE is an edit-type file located on the BA*xx*AFT subvolume, where *xx* is the number of the current release.

Product Indicator Table File (PITABLE)

The PITABLE has two purposes in a BASE24 system—it defines the entries displayed on the BASE24 Virtual Menu and it defines the segments included in segmented files. The Product Indicator Table (PI-TABLE) within the PITABLE specifies the entries displayed on the BASE24 Virtual Menu for a SUPER/SUPER user. The Segment Table (SEG-TABLE) in the PITABLE specifies the file product segments used for the logical network. Both tables allow for up to 256 entries.

The PITABLE delivered with BASE24 software contains default entries which must be modified when a BASE24 system is installed to contain only the entries needed for your system. A product can have one entry on the BASE24 Virtual Menu and one segment. A product cannot have multiple BASE24 Virtual Menu entries and multiple segments. A product can have a segment and not have an entry on the BASE24 Virtual Menu, and vice-versa. For example, the BASE24-atm Non–Currency Dispense add-on product can have an entry in the Segment Table, but not in the Product Indicator Table.

Refer to the ***BASE24 Classic 6.0 Installation Guide*** for detailed information on editing the PITABLE.

How File Segmentation Works

For illustrative purposes, consider the fictitious segmented file XYZ. By the nature of segmented file structures, XYZ is shared by a number of BASE24 products. If an institution uses only the BASE24-pos product, each XYZ record consists of a Base segment (which must be present for any BASE24 system) and a BASE24-pos segment.

Consider that the Base segment of an XYZ record is 600 bytes in length, and the BASE24-pos segment of an XYZ record is 550 bytes in length. In this case, the total length of an XYZ record would be 1150 bytes, as shown in the following diagram.

XYZ Record

BASE	BASE-pos
---------- 600 bytes ----------	---------- 550 bytes ----------
---------- a total of 1150 bytes ----------	

If BASE24-atm is integrated with the institution's BASE24-pos stand-alone system, a BASE24-atm segment is added to every XYZ record. The BASE24-atm segment contains those fields used exclusively by the BASE24-atm product. Both BASE24-pos and BASE24-atm products access the information that exists in the Base segment.

Now, suppose that in the previous example, the BASE24-atm product is added to the institution's system, and the BASE24-atm segment of an XYZ record is 350 bytes in length.

When the BASE24-atm product is added to the institution's BASE24 system, the BASE24-atm segment of the XYZ is added to each record, and the total length of an XYZ record is 1500 bytes. The new record structure is shown below.

XYZ Record

BASE	BASE24-atm	BASE-pos
------- 600 bytes -------	------- 350 bytes -------	------- 550 bytes -------
---------- a total of 1500 bytes ----------		

If the BASE24-pos system were integrated with BASE24-teller, instead of BASE24-atm, the BASE24-teller segment would be added to every XYZ record. The XYZ records would then consist of the Base segment, a BASE24-pos segment, and a BASE24-teller segment.

In this case, consider that the BASE24-teller segment of an XYZ record is 650 bytes in length. When the BASE24-teller product is added to the institution's BASE24 system, the total length of an XYZ record is 1800 bytes, as shown in the following diagram.

XYZ Record

BASE	BASE24-pos	BASE-teller
------- 600 bytes -------	------- 550 bytes -------	------- 650 bytes -------
---------- a total of 1800 bytes ----------		

If the BASE24-pos system were integrated with both BASE24-atm and BASE24-teller, the XYZ records would consist of the Base segment, a BASE24-atm segment, a BASE24-pos segment, and a BASE24-teller segment. When the BASE24 products are added to the institution's BASE24 system, the total length of an XYZ record would be 2150 bytes, as illustrated below.

XYZ Record

BASE	BASE24-atm	BASE24-pos	BASE-teller
---- 600 bytes ----	---- 350 bytes ----	---- 550 bytes ----	---- 650 bytes ----
---------- a total of 2150 bytes ----------			

Thus, segmenting a file allows for multiple product support in a flexible, and economical, file architecture.

Segments

BASE24 allows for up to 256 segments per record. The segments that are currently defined are identified in the table below. Note that segments are numbered starting from 01.

Segment Number	Segment Name	Description
01	Base	Carries information considered *common* to all applications that use the file and also contains the bit map key to the other segments. The Base segment is present in all segmented files.
02	BASE24-atm	Carries information used exclusively by the BASE24-atm product.
03	BASE24-pos	Carries information used exclusively by the BASE24-pos product.
04	BASE24-teller	Carries information used exclusively by the BASE24-teller product.
05–08		Reserved for future use.
09	BASE24-from host maintenance	Carries information used exclusively by the BASE24-from host maintenance product.
10	Europay, MasterCard, and Visa (EMV)	Carries information used exclusively by the BASE24-atm and BASE24-pos EMV add-on products.
11		Reserved for future use.
12	BASE24-mail	Carries information used exclusively by the BASE24-mail product.
13	BASE24-card	Carries information used exclusively by the BASE24-card product.
14		Reserved for future use.
15	BASE24-telebanking	Carries information used exclusively by the BASE24-telebanking product.

Segment Number	Segment Name	Description
16	PBF Credit Line	Carries credit line account information in the Positive Balance File (PBF).
17	PBF Customer Short Name	Carries customer short name information in the Positive Balance File (PBF).
18	BASE24-atm self-service banking (SSB) Base application	Carries information used exclusively by the BASE24-atm self-service banking (SSB) Base application.
19	BASE24-atm self-service banking (SSB) Check application	Carries information used exclusively by the BASE24-atm self-service banking (SSB) Check application.
20	BASE24-pos Address Verification	Carries information used exclusively by the BASE24-pos add-on Address Verification product.
21		Reserved for future use.
22	BASE24 Customer Service	Carries information used exclusively by the BASE24 Customer Service product.
23	PBF Preauthorization Hold	Carries information about preauthorized hold transactions. Only BASE24-pos creates preauthorized hold information; however, BASE24-atm and BASE24-teller use the preauthorized hold information when authorizing transactions.
24	Non–Currency Dispense	Carries information used exclusively by the BASE24-atm Non–Currency Dispense add-on product.
25	Stored Value	Carries information used exclusively by the BASE24-pos Stored Value add-on product.
26–31		Reserved for future use.

Segment Number	Segment Name	Description
32	CAF Account Information and Interchange Data	Carries account information in the Cardholder Authorization File (CAF) and interchange data.
33–59		Reserved for future use.
60–69		Reserved for use by ACI Americas development.
70–79		Reserved for use by ACI Asia/Pacific development.
80–89		Reserved for use by ACI Europe/Middle East/Africa development.
90–99		Reserved for customer use.
100–159		Reserved for future use.
160–169		Reserved for use by ACI Americas development.
170–179		Reserved for use by ACI Asia/Pacific development.
180–189		Reserved for use by ACI Europe/Middle East/Africa development.
190–199		Reserved for customer use.
200–256		Reserved for future use.

Note: Each segmented file supports a different subset of segments, depending on the product- or function-specific information in the file. For example, the CAF can contain shared information and product-specific information for the following BASE24 products, add-on products, and functions:

BASE24-atm
BASE24-atm self-service banking (SSB) Base application
BASE24-atm self-service banking (SSB) Check application
BASE24-atm Non–Currency Dispense
BASE24-card

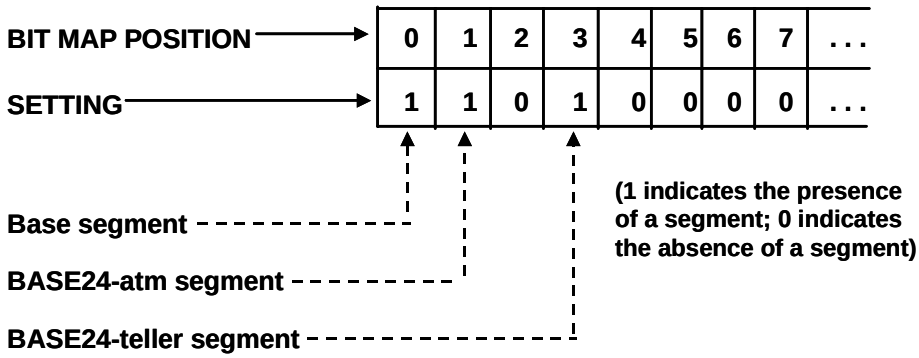
BASE24-pos
BASE24-pos Address Verification
Account information
Europay, MasterCard, and Visa (EMV)
Preauthorized holds

Thus, the CAF supports segments 0, 1, 2, 9, 12, 17, 18, 19, 22, 23, and 31. As another example, the Negative Card File (NEG) contains shared information and product-specific information for the BASE24-atm self-service banking Base and Check application add-on products. Thus, the NEG supports segments 0, 17, and 18 only.

Mapping the Segments

BASE24 products determine the segments present in authorization files by using standard fields at the beginning of the base segment of every Institution Data File (IDF) record. The FIID-SEG-MAP field contains a 32-position bit map listing of the first 32 segments. The FIID-SEG-MAP-EXT field contains a a 224-position bit map listing of next 224 segments (i.e., segments 33–256). These fields determine the segments present in the Cardholder Authorization File (CAF), Negative Card File (NEG), Positive Balance File (PBF), and Usage Accumulation File (UAF). For other segmented files, BASE24 products use segmentation utilities to determine the segments present.

For example, if an institution uses only the BASE24-atm and BASE24-teller products, the FIID-SEG-MAP bit map for each IDF record is set as follows:



In this example, all positions in the FIID-SEG-MAP-EXT bit map would be set to 0. The corresponding data records in the authorization files would be constructed as follows:

Data Record

Base Segment	BASE24-atm Segment	BASE24-teller Segment

Segment Size

The size of each segment is stored in a standard field at the front of the segment. The size field stored in the base segment contains the length of the base segment. Each BASE24 product segment also contains a similar length indicator field. Thus, when a record is read from disk, the length of each segment is known, and the BASE24 system can position to any point or segment in the record.

Template Files

A number of files are created routinely by a BASE24 system. For example, some files, such as the Online Maintenance File (OMF), transaction log files, and the Report File (RPT), are created daily by a BASE24 system. Other files, such as the Positive Balance File (PBF), the Cardholder Authorization File (CAF), and the Negative Card File (NEG), must be recreated during full-file refreshes.

A BASE24 system uses *template files* to create and recreate files. Template files contain no records; they are used simply to determine attributes for each of the files that must be created. Template files must reside on the same volume as the files created from them. The recommended name of the subvolume in which template files reside is *xxxxTPLT* (where *xxxx* indicates the name of the logical network).

For example, if a Cardholder Authorization File (CAF) called $DATA.PRO1DATA.CAF exists, then a template file called $DATA.PRO1TPLT.CAF also exists. The CAF on the subvolume PRO1DATA contains the actual records for each of the institution's cardholders. Periodically, the CAF must be completely updated, or refreshed. When the CAF is completely refreshed, the BASE24 Enscribe Refresh process uses the template CAF on the subvolume PRO1TPLT to recreate the new CAF on the subvolume PRO1DATA.

The template file is used as a model for establishing the attributes of the new file. The extent sizes, the number of extents allowed, and other file management considerations are not dictated by BASE24 code. Instead, the file attributes are specified by the settings for the template file. The template files allow file attributes to be easily adjusted to meet the specific needs of each individual institution.

BASE24 processes use internal calls to the Compaq NonStop operating system to access template files and to determine the characteristics of the file to be created by a BASE24 system.

The example that follows shows a partial listing of file characteristics for a Transaction Log File (TLF) template file.

```
$DATA.PRO1TPLT.TLYYMMDD  yy/mm/dd hh:mm
TYPE E
EXT ( 16 PAGES, 16 PAGES )
REC 4072
BLOCK  4096
ALTKEY ( “CR“, FILE 0, KEYOFF 40, KEYLEN 27, NO UPDATE  )
ALTKEY ( “TR“, FILE 0, KEYOFF 16, KEYLEN 24, NO UPDATE  )
ALTFILE ( 0, \B24.$DATA.PRO1TPLT.T0YYMMDD )
REFRESH
MAXEXTENTS 16
OWNER 108,255
SECURITY (RWEP): GGGO
DATA MODIF: mm/dd/yy  hh:mm
EOF 0 (0.0% USED)
EXTENTS ALLOCATED: 0
```

Using the TLF template file, any characteristic of the file can be modified. In this example, the extent size could be increased, if necessary, using the Compaq NonStop File Utility Program (FUP). A change to the characteristics of the file takes effect when the next TLF is created. File characteristics such as file type and record size should not be modified. Note that for single-partition refreshing of a partitioned file, the template file and the partitioned file must have the same file characteristics.

For each file that has a template file associated with it, the fully qualified name of the template file must be specified in the Logical Network Configuration File (LCONF). The template file is named in the same LCONF assign that specifies the fully qualified file name. For example, the template file for the POS Transaction Log File (PTLF) is named in the POS-PTLF assign.

File and Table Documentation

BASE24 files and record structures are documented online using comments in the Data Definition Language (DDL) source schema files that define the file or record structure. BASE24 tables are documented online using comments either in DDL source schema files or in SQL conversational interface (SQLCI) input files.

DDL Source Schema Files

Information on the use and content of each file field or table column precedes the definition of the field or column in DDL source schema files. Each line of description is marked by an asterisk (*). For example, the following is how the PREFIX field in the Card Prefix File (CPF) is documented in the DDL source schema file named DDLFCPF:

```
*
*             The card prefix to which this record information
*             applies.  The value in this field is used to identify
*             the FIID, Track 2 offsets, and authorization criteria
*             for cardholders whose card prefix numbers match the
*             value contained in this field.
*
     04 PREFIX                    TYPE BINARY 64.
```

To improve the readability and usability of the online information, ACI provides a utility called DDLDOC. The DDLDOC utility retrieves information from the compiled data dictionary and formats the information as the user requests. The information provided by the DDLDOC utility includes the field or column position in the file or table, the field or column name, and the data type of the field or column. Field and column names can be shown in COBOL or TAL format. Detailed field or column descriptions, when requested by the user, are printed *below* the field or column name and data type information. For example, the same PREFIX field information shown above appears as follows when formatted by the DDLDOC utility for COBOL output:

```
7-14      04  PREFIX                              TYPE BINARY 64 SIGNED.

              The card prefix to which this record information
              applies.  The value in this field is used to identify
```

```
                the FIID, Track 2 offsets, and authorization criteria
                for cardholders whose card prefix numbers match the
                value contained in this field.
```

For more information on DDL source schema files, refer to the appropriate Compaq NonStop documentation. The DDLDOC utility is described in section 2.

SQLCI Input Files

Some BASE24 tables are described in SQLCI input files rather than DDL source schema files. In these SQLCI input files, information on the use and content of each column precedes the definition of the column. Each line of description is marked by two hyphens (--). For example, the following is how the Customer Table (CSTT) PVD column is documented in the SQLCI input file named CSTTRS:

```
-- comment
-- comment    The PIN Verification Digits used in the PIN
-- comment    verification process.  The value in this column is
-- comment    based on the type of PIN verification method used
-- comment    and is required if the value is to be maintained
-- comment    in the Customer Table.
-- comment
-- comment    This column can contain spaces or hexadecimal values
-- comment    with no leading or embedded spaces.  The default value
-- comment    is spaces.
-- comment

  PVD                  CHAR (16) UPSHIFT  DEFAULT SYSTEM  NOT NULL,
```

SQLCI input file comments denoted by hyphens are not stored in the COMMENTS table on the SQL system catalog.

DDL Comments

For the most part, comments are printed in DDLDOC output *exactly* as they appear in the DDL source schema files. The DDLDOC utility strips the asterisk (*) and any leading spaces off of the left hand side, and aligns the comments with the field name. However, lines of text are not “joined” if they are too short and are not broken if they are too long. To allow for printing comments on an 80-column printer, the comments should not be more than 56 characters per line.

Stripping Leading Spaces

The DDLDOC utility determines the number of leading spaces to strip off of each line of comments by the number of leading spaces on the first line of comments in the comment block. A comment block is a series of consecutive lines marked as being comments with an asterisk (*). If the first line of comments in the block has 10 leading spaces, the DDLDOC utility strips the first 10 characters off of each line. For example, consider the following sample comment block. The numbers across the top mark the number of characters; they are provided for illustrative purposes only.

```
         1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
*                    The ZIP code for the cardholder's billing address.
*                The ZIP code must be five or nine digits long. If the ZIP
*                code is five digits long, it is left-justified and blank-filled.
```

The first line of the comment block has 19 leading spaces, so, for this comment block, the DDLDOC utility would strip the asterisk plus 19 characters off of the beginning of each line. After the DDLDOC utility has stripped off the leading characters, the comment block above would appear as follows:

```
         1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
The ZIP code for the cardholder's billing address.
IP code must be five or nine digits long. If the ZIP
is five digits long, it is left-justified and blank-filled.
```

Because the DDLDOC utility uses the number of leading spaces on the first line of a comment block to determine the number of characters to strip off each line in the block, the first line of comment information should not be indented more than any subsequent line. The DDLDOC utility recalculates the number of characters to strip off each time it reads a new comment block (e.g., for each field description).

Formatting Comment Text

After leading spaces are stripped off of each line, the DDLDOC utility retrieves the comment text for each line. The comment text is placed in a 56 character per line buffer. If the line of comments contains more than 56 characters, the line is truncated on the right at 56 characters. The result of this operation on the text from the example is as shown below:

```
         1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
The ZIP code for the cardholder's billing address.
IP code must be five or nine digits long. If the ZIP
is five digits long, it is left-justified and blank-fill
```

The comments retrieved from the dictionary are placed in the output so that they are indented 14 characters from the beginning of text on the left (that is, 14 characters in addition to the left margin). This results in a total comment line length of 70 characters, as shown below:

```
         1         2         3         4         5         6         7         8
12345678901234567890123456789012345678901234567890123456789012345678901234567890
              The ZIP code for the cardholder's billing address.
              IP code must be five or nine digits long. If the ZIP
              is five digits long, it is left-justified and blank-fill
```

ACI has formatted the comments in its DDL source schema files for most file and record structures so that comments do not extend beyond column 70 (that is, so that the comments themselves are not more than 56 characters per line).

Allowing All Comments to Print

If you are requesting DDLDOC output for a structure where the comments are longer than 56 characters per line, a DDLDOC formatting option allows you to specify that the comments should be allowed to extend beyond column 70 (not including any left margin). When this option is used, the DDLDOC utility retrieves and formats all of the characters for each line of comments.

Note: Even with the formatting option specifying that comments should not be truncated, it is possible that comments could be truncated on the right by the printer. For example, if the comments in the DDL source schema are 70 characters in length and the left margin is set to 10 characters wide, the total comment line length is 94 characters (10 character margin + 14 character indent from the margin + 70 characters of text). If the output is sent to an 80 column printer, 14 characters of comments will be truncated on the right.

Comment Guidelines

If you are writing comments in a DDL source schema file and you plan to use the DDLDOC utility to retrieve comments from the compiled dictionary, ACI recommends that you follow the guidelines below:

1. Limit the actual text for each line of comments to no more than 56 characters.
2. Line up the beginning of each line of comments with the first line of comments in the block. If desired, individual lines can be indented more than the first line of comments, but do not indent the first line more than subsequent lines.

DDL Constants

Besides file and record structures, the DDLDOC utility also allows you to view constants from a compiled data dictionary. The DDLDOC utility retrieves information from the compiled data dictionary and displays it in the format requested by the user. The information provided by the DDLDOC utility includes the constant's name, data type (if known), and value.

For example, the following is an example of the DDLDOC output for the FAPF-VER-JAN95-L constant compiled in the data dictionary on the OC*xx*DICT subvolume, where *xx* is the number of the current release.

```
** Unknown Type ** Constant FAPF-VER-JAN95-L: 1000
```

DDL Locations

BASE24 DDL source schema files and data dictionaries are located on various subvolumes, depending on the BASE24 product and module or object with which the file or table is associated. The standard location for each file or table data dictionary or DDL source schema file is provided in section 3.

SQLCI Input File Locations

BASE24 SQLCI input files are located on various subvolumes, depending on the BASE24 product and module or object with which the file is associated. These files are used to create SQL tables during installation or during full-file refresh. The standard location for each table's SQLCI input file is provided in section 3.

Formal File and Record Structure Documentation

Some DDL structures, for such things as tokens, transaction messages, and refresh and extract tape header and trailer records, are documented in formal publications rather than using the DDLDOC utility. This formal documentation is described below. In addition, the DDLDOC output for refresh and extract records is provided on the BASE24 Product Documentation CD-ROM.

Token Documentation

Tokens used by BASE24 products are documented in the ***BASE24 Tokens Manual***.

Internal Message Documentation

Internal messages used by BASE24-atm, BASE24-pos, BASE24-teller, and BASE24-telebanking products are documented as follows in the transaction processing manuals for the respective products.

- The BASE24-atm Standard Internal Message (STM) is documented in the ***BASE24-atm Transaction Processing Manual***.
- The BASE24-pos Standard Internal Message (PSTM) is documented in the ***BASE24-pos Transaction Processing Manual***.
- The BASE24-telebanking Standard Internal Message (BSTM) and Internal Transaction Data (ITD) are documented in the ***BASE24 Remote Banking Transaction Processing Manual***.
- The BASE24-teller Standard Internal Message (TSTM) is documented in the ***BASE24-teller Transaction Processing Manual***.

ISO External Message Documentation

ISO external messages as used by the ISO Host Interface and From Host Maintenance processes are documented in the ***BASE24 External Message Manual***.

ISO external messages as used by BIC ISO Interchange Interface processes are documented in the ***BASE24 BIC ISO Standards Manual***.

Refresh and Extract Record Documentation

Refresh and extract tape headers, tape trailers, and organizational record formats are documented in the ***BASE24 Refresh and Extract Operators Manual***. All other refresh and extract record structures are documented using the DDLDOC utility. The printed DDLDOC output for refresh and extract records is provided on the BASE24 Product Documentation CD-ROM.

File and Table Maintenance Documentation

File and table structures documentation created using the DDLDOC utility is not intended for basic file and table maintenance functions (that is, it is not intended to be used when accessing BASE24 CRT access screens). BASE24 file and table maintenance screen fields are documented in the ***BASE24 Base Files Maintenance Manual*** and in the various product-specific files and table maintenance manuals.

ACI Worldwide Inc.

Section 2

The DDLDOC Utility

The DDLDOC utility reads Compaq NonStop Data Dictionaries and returns information from the dictionaries to the user. The DDLDOC utility is run to request information from the dictionaries about a DDL definition, record structure, or constant. This section describes how to request data dictionary information and provides examples of DDLDOC output.

DDLDOC Syntax

The syntax for running the DDLDOC utility is described below. Items enclosed in square brackets ([ ]) are optional. Items not enclosed in square brackets are required and must be entered exactly as they appear. Italicized items are variables and are defined in the paragraphs that follow.

Syntax

[RUN] DDLDOC [/ [IN *in-file*] [, OUT *output-location*] /] [[*option*] [, *option*]...]

IN *in-file* — The name of a DDLDOC input file. Input files can contain any DDLDOC command or series of commands. Input files can be used to request a series of record structures, definitions, or constants. For more information on creating and using input files, refer to the topic "Creating and Using Input Files" later in this section.

OUT *output-location* — The output location to which DDLDOC output should be sent. This field can contain the name of a printer, any valid Spooler location, or any valid file name. If a file name is specified, the file must not currently exist. The *output-location* variable is optional in the command syntax. If an *output-location* is not specified, DDLDOC output is displayed on the terminal from which the run command was executed.

option — A DDLDOC option defining the output that is required from the DDLDOC utility. Multiple options can be entered to specify and control the output that a user wants to see. There are three groups of options: those that format output, those that request information from the dictionary, and those used for other purposes. The options in each of these groups are described on the following pages.

Any option can be entered with or without the initial DDLDOC prompt character—a question mark (?). For example, ?RECS and RECS are both valid options. When multiple options are entered on the command line, each option is separated from the next by a comma.

Note: Format options must be entered on the DDLDOC command line before informational request options and remain in effect until the DDLDOC utility run completes or until subsequent format options are encountered that change the format.

Example

```
DDLDOC COBOL,DETAIL,SHOW CAF,NODETAIL,TAL,SHOW CPF
```

The example above requests a detailed, COBOL-formatted listing of the CAF record structure, and a TAL-formatted listing without comments for the CPF record structure. Because no output location was specified, the information is displayed on the terminal where the command was entered.

Example

```
DDLDOC/OUT $S.#HOLD/
```

The example above starts the DDLDOC utility interactively. Any output requested during the session is sent to the spooler location $S.#HOLD.

Example

```
DDLDOC/IN FRAT, OUT $S.#HOLD/
```

The example above requests the information defined in the input file named FRAT. The output is sent to the spooler location $S.#HOLD.

DDLDOC Command Options That Format Output

The following DDLDOC command options specify the appearance of the DDLDOC output. Format options must be entered on the DDLDOC command line before informational request options and remain in effect until the DDLDOC utility run completes or until subsequent format options are encountered that change the format.

In some cases, more than one literal can be used to indicate the same DDLDOC output. In these cases, the literals are shown side by side, separated by a vertical bar. For example, COBOL | NOTAL indicates that COBOL and NOTAL produce the same results. Only one form of an option needs to be entered in the DDLDOC command.

Option	Description
COBOL | NOTAL	Requests COBOL format output for records and definitions. Only one form of the option needs to be entered. For more information on what COBOL format output looks like, refer to the topic “Sample COBOL Output” later in this section. COBOL format output is canceled by the TAL option. This option has no impact on the format output for constants. Default: TAL
DETAIL [ON]	Requests any detailed field descriptions (comments) that were compiled into the dictionary. Only one form of the option needs to be entered. Although the comments in the DDL source schema file appear before the field name, the DDLDOC utility prints comments after the field name. Detailed field descriptions are canceled by the DETAIL OFF or NODETAIL options. **Note:** If comments were not compiled into the dictionary, the DDLDOC utility will not be able to retrieve any comments. Comments are compiled into the dictionary using the ?COMMENTS DDL directive. Refer to the Compaq NonStop documentation on compiling data dictionaries for more information on including comments in the dictionary. Default: NODETAIL

Option	Description
DETAIL OFF | NODETAIL	Indicates that the DDLDOC utility should not print any comments that were compiled into the dictionary. Only one form of the option needs to be entered. This is the default. Comments are requested using the DETAIL or DETAIL ON options. Default: NODETAIL
LMARGIN *margin*	Sets the left margin for the output. Valid values for *margin* range from 0 through 15. The default left margin is 0. A subsequent LMARGIN option specifying a different *margin* changes the left margin. Margins should be specified before other options that format output. **Note**: The DDLDOC utility has a comment line length of 70 characters (14 character indent plus 56 characters of comments). When setting the left and right margins, it is important to maintain at least 70 characters between the two margins (for example, if the left margin is set to 7, the right margin should be set to at least 77). If the distance between the margins is less than 70 characters and detailed output is requested, comments may be truncated on the right. Default: LMARGIN 0
RAGGED OFF	Indicates that the DDLDOC utility should truncate any comments for which the number of characters in the source file comment line exceeds 56 characters. This is the default. Truncation of comments beyond column 56 is disabled by the RAGGED ON option. Default: RAGGED OFF

Option	Description
RAGGED ON	Indicates that the DDLDOC utility should print any comments that are longer than 56 characters in the DDL source schema file. By default, when the DDLDOC utility retrieves comments from the dictionary, it retrieves up to 56 characters per line of comments. These comments are placed in the output so that they are indented 14 characters from the beginning of text on the left (that is, 14 characters in addition to the left margin). This results in a comment line length of 70 characters. ACI has formatted the comments in the DDL source schema files for most file and record structures so that comments are not longer than 56 characters per line. However, this option allows comments that are longer than 56 characters to be printed, within the constraints of the output device and the left and right margins. Default: RAGGED OFF
RMARGIN *margin*	Sets the right margin for the data type information only (in general, comments do not extend beyond column 70; refer to the RAGGED ON option for more information about margins). Valid values for *margin* range from 60 through 132. The default right margin is 70. A subsequent RMARGIN option specifying a different *margin* changes the right margin. Margins should be specified before other options that format output. **Note:** The DDLDOC utility has a comment line length of 70 characters (14 character indent plus 56 characters of comments). When setting the left and right margins, it is important to maintain at least 70 characters between the two margins (for example, if the left margin is set to 7, the right margin should be set to at least 77). If the distance between the margins is less than 70 characters and detailed output is requested, comments may be truncated on the right. Default: RMARGIN 70

Option	Description
TAL	Requests TAL format output for records and definitions. TAL format output is the default. For more information on what TAL format output looks like, refer to the topic “Sample TAL Output” later in this section. TAL format output is canceled by the COBOL or NOTAL options. This option has no impact on the format output for constants. Default: TAL

DDLDOC Command Options That Request Information

The following DDLDOC command options request the information that appears in DDLDOC output.

In some cases, more than one literal can be used to indicate the same DDLDOC output. In these cases, the literals are shown side by side, separated by a vertical bar. For example, FILE OFF | NOFILE indicates that FILE OFF and NOFILE produce the same results. Only one form of an option needs to be entered in the DDLDOC command.

Filters

The CONS, DEFS, RECS, and SHOW output command options allow the command to be qualified through the use of an optional filter. A filter is a literal or a combination of literal and wildcard characters that limit the objects affected by the command. Only those objects that match the filter are included in the command output.

A literal is the full or partial name of name of a constant, definition, or record object. Wildcard characters include the asterisk (*) and question mark (?). The asterisk denotes a variable-length text string in the object name. The question mark denotes a single character in the object name.

For example, to display a list of constants that end in -L only, you would use the filer *-L with the CONS command. To display a list of constants that begin with IDF- followed by any three characters, and that end with -L only, you would use the filter IDF-???-L filter with the CONS command. To display the value and type of a known constant, you would use the full constant name literal with the SHOW

CONS command to limit the output to the specified constant. For example, to display the output for the RS-NO-ERR-L constant, you would use the literal RS-NO-ERR-L literal with the SHOW CONS command.

Any combination of wildcard and literal characters can be used in a filter, as long as the length of the filter does not exceed the Compaq NonStop limit of 30 characters for DDL names.

Option Descriptions

The following table describes the function and syntax for each command that request information appearing in DDL output.

Option	Description
CONS [*filter*]	Requests a list of the names of all constants in the current dictionary or only the constants in the current directory that match a specified filter. A constant is any data structure defined in the DDL source schema file with the keyword CONSTANT. Typically, constants define a fixed data value that is used frequently in other data structures. For example, the constant OBJ-ID-ANS-L defines the object ID value for the Application Network Services object as 1. Default: Not applicable
DEFS [*filter*]	Requests a list of the names of all definitions in the current dictionary or only the definitions in the current directory that match a specified filter. A definition is any data structure defined in the DDL source schema file with the keyword DEFINITION. Typically, definitions define a data structure that is used frequently in other data structures. For example, the definition LAST-FM is used in each file to provide information about the last file maintenance done to the file. Default: Not applicable

Option	Description
FILE [ON]	Requests file information from the DICTKDF and DICTRDF to be included when displaying record definitions with the SHOW option. These files are created when the dictionary is compiled. The information returned from the DICTKDF and DICTRDF includes the file name, record length, file type, file code, maximum extents, primary and secondary extents, and key information. File information is returned for each record structure until the DDLDOC utility run completes or until a subsequent FILE OFF or NOFILE option is encountered. Default: NOFILE
FILE OFF | NOFILE	Cancels a previous FILE request. Only one form of the option needs to be entered. Default: NOFILE
FILLER [ON]	Requests a listing of only those fields for which the field name begins with the literal FILLER when displaying records or definitions with the SHOW option. For example, this option causes fields named FILLER, FILLER1, or FILLER-A to be listed, but not a field named A-FILLER. This option can be used to locate fields that are not currently being used to store data. The FILLER option cancels the DETAIL option, and is canceled by a subsequent DETAIL option. Default: NOFILLER
FILLER OFF | NOFILLER	Cancels a previous FILLER request. Default: NOFILLER
HELP	Requests the DDLDOC help facility. The help provided describes DDLDOC options. For more information on help, refer to the topic “The DDLDOC Help Facility” later in this section. Default: Not applicable

Option	Description
LGTHONLY [ON]	Requests only the length of entire record or definition structures. When this option is requested, the DDLDOC utility returns the length of the structure named in the SHOW option, but no other information about the structure. The LGTHONLY option cancels the DETAIL option, and is canceled by a subsequent DETAIL option. Default: NOLGTHONLY
LGTHONLY OFF | NOLGTHONLY	Cancels a previous LGTHONLY request. Default: NOLGTHONLY
LGTH02 [ON]	Requests only the individual and total lengths of level 02 fields in record or definition structures named in the SHOW option. The total length includes all fields defined below the 02 structure. The LGTH02 option cancels the DETAIL option, and is canceled by a subsequent DETAIL option. Default: NOLGTH02
LGTH02 OFF | NOLGTH02	Cancels a previous LGTH02 request. Default: NOLGTH02
RECS [*filter*]	Requests a list of the names of all records available in the current dictionary or only the records in the current directory that match a specified filter. A record is any data structure defined in the DDL source schema file with the keyword RECORD. Default: Not applicable
SHOW [CON | DEF | REC] [*filter*]	Requests that the DDLDOC utility display the indicated constants, definitions, or records in the dictionary. The optional CON, DEF, or REC syntax limits the filter to constants, definitions, or records, respectively. A filter must be included with this command. Default: Not applicable

Option	Description
USERFLD	Requests a listing of only those fields for which the field name begins with the literal USER-FLD when displaying records or definitions with the SHOW option. For example, this option causes fields named USER-FLD, USER-FLD1, or USER-FLD-10A to be listed, but not a field named A-USER-FLD). This option can be used to locate fields that are not currently being used to store data. The USERFLD option cancels the DETAIL option, and is canceled by a subsequent DETAIL option. Default: NOUSERFLD
USERFLD OFF | NOUSERFLD	Cancels a previous USERFLD request. Default: NOUSERFLD

Miscellaneous DDLDOC Command Options

The following miscellaneous DDLDOC command options are also available.

Option	Description
! [*num* | *text*]	Reexecutes a previous command. The command is retrieved from the command buffer and executed again. You can use the ! command on the last command entered, on a command number, or on a text string. The DDLDOC utility buffers the last 20 commands entered, along with an associated command number. Only commands in the buffer can be referenced by the ! command. A command number (*num*) or text string (*text*) are optional with this command. If a command number or text string is not entered, the last entered command is reexecuted. If a command number is entered, the command associated with that number in the buffer is reexecuted. Valid values for *num* are from 1 to 9999. If a text string is entered, the command buffer is searched for a command that matches the entered string for the length of the entered string. The search is not case sensitive and discards leading spaces. If two or more commands in the buffer match the entered string, the most recently entered command is reexecuted. The HISTORY command displays the current contents of the command buffer. Default: Not applicable
COMMENT OFF	Cancels a previous COMMENT ON request. Default: COMMENT OFF

Option	Description
COMMENT ON	Indicates that the DDLDOC utility should print any comments contained in the input file to the output location. Comments in the input file are identified by the hyphen character (-) in the first position on the line. Default: COMMENT OFF
ERROR OFF	Indicates that the DDLDOC utility should not produce error messages when it receives an invalid command. Default: ERROR ON
ERROR ON	Indicates that the DDLDOC utility should print an error message if it receives an invalid command. The error message is sent to the home terminal and to the output location. Default: ERROR ON
EXIT	Indicates that the DDLDOC utility should stop running. This option is used to exit the DDLDOC utility when it is started interactively. For more information on running the DDLDOC utility interactively, refer to the topic “Running The DDLDOC Utility.” Default: Not applicable

Option	Description
FC [*num* | *text*]	Provides standard Compaq NonStop Fix Command processing. You can use the FC command on the last command entered, on a command number, or on a text string. The DDLDOC utility buffers the last 20 commands entered, along with an associated command number. Only commands in the buffer can be referenced by the FC command. A command number (*num*) or text string (*text*) are optional with this command. If a command number or text string is not entered, the last entered command is displayed. If a command number is entered, the command associated with that number in the buffer is displayed. Valid values for *num* are from 1 to 9999. If a text string is entered, the command buffer is searched for a command that matches the entered string for the length of the entered string. The search is not case sensitive and discards leading spaces. If two or more commands in the buffer match the entered string, the most recently entered command is displayed. The HISTORY command displays the current contents of the command buffer. Default: Not applicable
HI[STORY]	Displays the contents of the command buffer. If fewer than 20 commands have been entered, only the commands entered are displayed. If more than 20 commands have been entered, only the last 20 commands entered are displayed. Default: Not applicable
PAGE	Indicates that the DDLDOC utility should start a new page of output. The DDLDOC utility starts a new page for each option that requests information. Default: Not applicable

Option	Description
VER[SION]	Displays the current dictionary version from the DICTDDF file. Default: Not applicable
VOLUME *location*	Specifies the location of the data dictionary. This option is used to access a dictionary on a different volume or subvolume from the current volume or subvolume. Default: Current volume and subvolume

Running The DDLDOC Utility

The DDLDOC utility can be run interactively or noninteractively. When the DDLDOC utility is run interactively, you can enter any option at the DDLDOC prompt. After each option is processed by the DDLDOC utility, the prompt reappears, so that you can enter the next option. When the DDLDOC utility is run noninteractively, all of the DDLDOC options are entered with the initial run command. When the DDLDOC utility has processed all of the options, the TACL prompt is redisplayed. The following paragraphs provide more information on running the DDLDOC utility.

Starting The DDLDOC Utility Interactively

The DDLDOC utility is started interactively if no option or no input file is specified in the run command. For example, the following commands start the DDLDOC utility interactively:

```
DDLDOC
DDLDOC/OUT output-location/
```

When either of these commands are entered at a TACL prompt, the DDLDOC prompt is displayed. The DDLDOC prompt is a question mark (?). Any DDLDOC option can be entered at this prompt by typing the appropriate command syntax and pressing the **Return** key. Valid DDLDOC options are described earlier in this section in the subsection on “DDLDOC Syntax.” The only difference in these two commands is the output location. The first command directs output to the terminal where the DDLDOC utility was started. With the second command, DDLDOC output is sent to the location specified by the *output-location* variable.

Stopping The DDLDOC Utility Interactively

To stop the DDLDOC utility when it is running interactively, enter the EXIT command option at the DDLDOC prompt or press the **Break** key at any time.

Running The DDLDOC Utility Noninteractively

To run the DDLDOC utility noninteractively, enter an input file or one or more options with the DDLDOC run command. When an input file or an option is specified in the run command, the DDLDOC utility stops after the input file or the option has been processed. For example, the following commands start the DDLDOC utility noninteractively:

```
DDLDOC/IN TBAT/COBOL
DDLDOC/OUT output-location/COBOL,DETAIL,SHOW CAF,SHOW NEG
```

The DDLDOC Help Facility

The DDLDOC utility has its own help information, which can be accessed by entering the HELP option on the command line. The help information defines the options that can be entered on the command line.

Command Entry

To request DDLDOC help, enter the following command at a TACL prompt, or enter HELP at the DDLDOC prompt:

```
DDLDOC HELP
```

Help Information Output

The DDLDOC utility returns the information shown below to both the screen and the output location when help information is requested.

```
MM/DD/YY HH:MM                                                PAGE:    1
Listing Help


DDLDOC - Program to extract DDL definitions for print formatting.

      The following commands are available at present:

  ?COBOL       - Produces COBOL output.
  ?COMMENT ON  - Output command file comments to listing.
  ?COMMENT OFF - Turns off the ?COMMENT command (default).
  ?CONS [<f>]  - Shows all CONSTANTS in the DDL file.  <f> is an
                 optional filter (e.g., *-L or CONST-???-L).
  ?DEFS [<f>]  - Shows all DEFINITIONS in the DDL file.  <f> is
                 optional filter (e.g., T* or ANS-OBJ-R???).
  ?DETAIL ON   - Shows the comments in the DDL source code (if
                 compiled in).
  ?DETAIL OFF  - Turns off the ?DETAIL command (default).
  ?ERROR ON    - Displays an error message for invalid commands
                 (default).
  ?ERROR OFF   - Turns off the ?ERROR command.
  ?EXIT        - Stops the run of DDLDOC.
  ?FC [<n>|<c>]- Fix Command.  <n> is optional command number
                 obtained from HIstory.  <c> is optional command
                 text (default is to fix previous command).
  ?![<n>|<c>]  - Repeat Command.  <n> is optional command number
```

```
                  obtained from HIstory.  <c> is optional command
                  text (default is to repeat previous command).
  ?FILE ON     - Shows information from the RDF and KDF files.
  ?FILE OFF    - Turns off the ?FILE command (default).
  ?FILLER ON   - Outputs all filler.
  ?FILLER OFF  - Turns off the ?FILLER command (default).
  ?HELP        - Shows this information.
  ?HI          - History of previous 20 commands.  Used in
                  conjunction with the ?FC and ?! commands.
  ?LGTHONLY ON - Outputs the length of the record only.
  ?LGTHONLY OFF- Turns off the ?LGTHONLY command (default).
  ?LGTH02 ON   - Outputs the length of the record and 02 level
                  structs (good for finding lengths of segments).
  ?LGTH02 OFF  - Turns off the ?LGTH02 command (default).
  ?LMARGIN <n> - Sets the left margin of the output to <n>
                  (default 0).
  ?NOTAL       - Produces COBOL output.
  ?PAGE        - Inserts a page break in the output file.
  ?RAGGED ON   - Ignore right margin for DDL comments.
  ?RAGGED OFF  - DDL comments truncated at right margin (default).
  ?RECS [<f>]  - Shows all RECORDs in the DDL file.  <f> is
                  optional filter (e.g, NAME* or ABC-????-*).
  ?RMARGIN <n> - Sets the right margin of the output to <n>
                  (default 70).
  ?SHOW <d><f> - Outputs rec/def/con <f>.  <d> is optional and
                  may be one of: CON, DEF, or REC. <f> may be an
                  exact match or a filter (see ?DEFS or ?RECS).
  ?TAL         - Produces TAL output (default).
  ?USERFLD ON  - Outputs all user^fld defs.
  ?USERFLD OFF - Turns off the ?USERFLD command (default).
  ?VER         - Displays the current dictionary version from DICTDDF.
MM/DD/YY HH:MM                                          PAGE:    2

Listing Help

  ?VOLUME <v>  - Sets the default volume to <v>.
```

DDLDOC Output

The remainder of this section provides samples of DDLDOC run commands and the output that is produced by the commands.

Each page of DDLDOC output has a standard heading. This heading provides information about the data dictionary from which the DDLDOC output was retrieved. The standard heading includes the following fields:

```
MM/DD/YY HH:MM      vol   subvol                              PAGE:    #
```

MM/DD/YY HH:MM — Indicates the date and time the data dictionary was compiled. By comparing the date and time printed on a page of DDLDOC output to the Compaq NonStop file date of the dictionary files, you can determine whether your printed copy contains the most current information.

vol subvol — Indicates the volume and subvolume of the data dictionary from which the information was retrieved.

PAGE: # — Indicates the page number of the output. Each page is numbered sequentially, starting with page 1.

Determining What Data Structures Are Available

The DDLDOC utility returns specific information from the data dictionary. In order to request that information, you must know the name of the structure that defines the information. One method of determining the data structures available in the dictionary is to request the information using the CONS, DEFS, or RECS options.

When you enter the CONS, DEFS, or RECS options at the DDLDOC prompt or include them on the command line or in an input file, the DDLDOC utility reads the data dictionary and retrieves the names for all data structures of the type requested (that is, constants, definitions, or records). This information is then printed in the DDLDOC output. The format for CONS, DEFS, and RECS output is the same.

Depending on the output line length (right margin minus left margin), the DDLDOC utility displays two or three columns of information. Each column of information is 30 characters wide, plus two blank characters between columns for spacing. If the output line length is less than 96 characters, the information is shown in two columns. If the output line length is 96 columns or greater, the information is shown in three columns.

Command Entry

To request information about the data structures included in the data dictionary, enter one or more of the following commands at the TACL prompt, or enter CONS [*filter*], DEFS [*filter*], or RECS [*filter*] at the DDLDOC prompt. The [*filter*] variable allows you to restrict the structures listed to those that match the text string of the filter.

```
DDLDOC CONS [filter]
DDLDOC DEFS [filter]
DDLDOC RECS [filter]
```

Sample RECS Output

A sample of the information returned by using the RECS option without a filter on the data dictionary in the BA*xx*DDL subvolume is shown below, where *xx* is the number of the current release.

```
Listing RECORDS defined in Dictionary

  ATT                           SAF
  CPF                           CAFV
  CAF                           ECF
  EMF                           ERF
  HCF                           IDF
  ICF                           ICFE
  SPF                           TKN
  ARF                           ILF
  KEY6                          KEYA
  KEYD                          KEYF
  KEYICC                        LCONF
  NEG                           OMF
  PBF                           PDF
  RFH                           RBH
  RBT                           RFT
  RGCF                          RPT
  SCLR                          SEC
  UFIR                          UPFR
```

```
STF                    SURF
TCF                    UAF
APCF                   IPCF
TESTDATA
```

Producing COBOL Record and Definition Structures

COBOL format output provides the same basic information that is entered in the source schema file, with the addition of field position information. Field names are provided in COBOL format—all uppercase, with hyphens separating descriptive parts of the name. A standard heading printed at the top of each page of COBOL output identifies the information that is returned. This standard heading is shown below, followed by descriptions of the information that appears below each heading:

```
POS      LVL  FIELD NAME AND DESCRIPTION                    DATA TYPE
```

POS — Identifies the position of the field in the record or definition structure. For a group field, the position column identifies the position of the group. When fields appear multiple times (because of an OCCURS clause), the positions in this column for an individual field are the positions for the first occurrence of the field.

LVL — Indicates the level number assigned to the field in the source schema file.

FIELD NAME AND DESCRIPTION — Contains the name of the field. This column also contains the detailed field description, if the DETAIL or DETAIL ON option was included on the command line.

DATA TYPE — Displays the picture clause assigned to the field in the DDLs. The data type identifies the size of the field and the type of the field. Examples of data type entries are as follows:

PIC 9	Denotes a numeric-type field
PIC X	Denotes an alphanumeric-type field
TYPE BINARY	Denotes a numeric field stored in binary form

Several data type entries that are assigned in the DDL source schema files are displayed differently when displayed by the DDLDOC utility. The differences include:

- Information from REDEFINES, OCCURS, and OCCURS DEPENDING ON clauses appears in the DDLDOC output in brackets, beginning roughly in the center of the FIELD NAME AND DESCRIPTION column.
- Information from TYPE *definition* or TYPE * clauses also appears in the DDLDOC output in brackets ([ ]), beginning roughly in the center of the FIELD NAME AND DESCRIPTION column. For example, if the field PRO-NAME is defined as TYPE SYM-NAME, the DDLDOC output for the field would be as follows:

```
xx-xx      04  PRO-NAME          [SYM-NAME]                    PIC X(16).
```

- Fields that are defined as TYPE BINARY (8, 16, 32, or 64) are, by default, considered to be signed. For these fields, the DDLDOC output includes the literal SIGNED. If a field defined as TYPE BINARY includes the UNSIGNED clause, the DDLDOC output includes the literal UNSIGNED.
- All PIC clauses indicate the data type (for example, X for alphanumeric) followed by the size of the field in parentheses. Thus, a field defined as PIC 9 in the source schema file appears as PIC 9(1) in DDLDOC output, and a field defined as PIC XX appears as PIC X(2).

Command Entry

To request COBOL format output, include the COBOL or the NOTAL option on the command line, as shown in the following examples:

```
DDLDOC/OUT $S.#HOLD/COBOL,SHOW REC UPFR
DDLDOC/OUT $S.#HOLD/NOTAL,SHOW DEF ITD-*
```

Sample COBOL Output

A sample of COBOL format output is shown below. The sample does not include field descriptions, which would appear below the field names in the output.

```
MM/DD/YY HH:MM      $PRO1   BAxxDDL     UPFR                PAGE:    1

POS      LVL  FIELD NAME AND DESCRIPTION                     DATA TYPE

RECORD UPFR

1-404     00  UPFR
1-18      02  PRIKEY
1-16      04  ALIAS           [ALIAS]                      PIC X(16).
17-18     04  PROD-ID                          TYPE BINARY 16 SIGNED.
19-20     02  NUM-FILES                      TYPE BINARY 16 UNSIGNED.
21-404    02  FILE-LIST       [OCCURS   96]
                               [OCCURS DEPENDING ON NUM-FILES]
21-24     04  FILE-ID                                      PIC X(4).
              END   [size  404]
```

Producing TAL Record and Definition Structures

TAL format output provides field names in TAL format—all lowercase, fully qualified field names with circumflexes (^) separating descriptive parts of the name. A standard heading printed at the top of each page of TAL output identifies the information that is returned. This standard heading is shown below, followed by descriptions of the information that appears below each heading:

```
Offset[Len]   Field Name and Description                  Data Type
```

Offset — Identifies the offset of each field from the beginning of the record or definition data structure. For fields that occur multiple times (because of an OCCURS clause), the offset in this column is the offset to the first occurrence of the field.

[Len] — Indicates the length of the field. For group fields, this column contains the combined length of all fields in the group.

Field Name and Description — Contains the name of the field. This column also contains the detailed field description, if the DETAIL or DETAIL ON option was included on the command line. If a field occurs multiple times, the range of occurrences is indicated following the field name (for example, file^list [0:31] indicates that the file^list field occurs 32 times).

Data Type — Displays the picture clause listed in the DDLs. The data type identifies the size of the field and the type of the field. Examples of data type entries are as follows:

PIC 9	Denotes a numeric-type field
PIC X	Denotes an alphanumeric-type field
TYPE BINARY	Denotes a numeric field stored in binary form

Several data type entries that are assigned in the DDL source schema files are displayed differently when displayed by the DDLDOC utility. The differences include the following:

- Information from REDEFINES, OCCURS, and OCCURS DEPENDING ON clauses appears in the DDLDOC output in brackets, beginning roughly in the center of the FIELD NAME AND DESCRIPTION column.
- Information from TYPE *definition* or TYPE * clauses also appears in the DDLDOC output in brackets, beginning roughly in the center of the FIELD NAME AND DESCRIPTION column. For example, if the field ALIAS is defined as TYPE *, the DDLDOC output for the field would be as follows:

```
0     [16]    prikey.alias.byte[0:15]
                                [ALIAS]                    PIC X(16).
```

- Fields that are defined as TYPE BINARY (8, 16, 32, or 64) are, by default, considered to be signed. For these fields, the DDLDOC output includes the literal SIGNED. If a field defined as TYPE BINARY includes the UNSIGNED clause, the DDLDOC output includes the literal UNSIGNED.
- All PIC clauses indicate the data type (for example, X for alphanumeric) followed by the size of the field in parentheses. Thus, a field defined as PIC 9 in the source schema file appears as PIC 9(1) in DDLDOC output, and a field defined as PIC XX appears as PIC X(2).

Command Entry

To request TAL format output, include the TAL option on the command line, as shown in the following example:

```
DDLDOC/OUT $S.#HOLD/TAL,SHOW REC UPFR
```

Note: TAL format is the default format for output. Therefore, TAL format output can also be requested by not specifying a format on the command line, as in the following example:

```
DDLDOC/OUT $S.#HOLD/SHOW UPFR
```

Sample TAL Output

A sample of TAL format output is shown below. The sample does not include field descriptions, which would appear below the field names in the output.

```
MM/DD/YY HH:MM      $PRO1   BAxxDDL     UPFR                PAGE:    1

Offset[Len]   Field Name and Description                     Data Type

RECORD UPFR

0     [404]
0     [18]    prikey
0     [16]    prikey.alias.byte[0:15]
                              [ALIAS]                       PIC X(16).
16    [2]     prikey.prod^id                    TYPE BINARY 16 SIGNED.
18    [2]     num^files                        TYPE BINARY 16 UNSIGNED.
20    [384]   file^list[0:95]
                             [OCCURS DEPENDING ON NUM^FILES]
20    [4]     file^list[0:95].file^id.byte[0:3]             PIC X(4).
              END   [size  404]
```

Producing Detailed Field Descriptions

When detailed output is requested, detailed field descriptions are provided below the field name, if comments were compiled into the dictionary.

Command Entry

To request detailed field descriptions, include the DETAIL or DETAIL ON option on the command line, as shown in the following examples:

```
DDLDOC/OUT $S.#HOLD/DETAIL,COBOL,SHOW UPFR
DDLDOC/OUT $S.#HOLD/DETAIL ON,SHOW REC UP*
```

Sample Detail Output

A sample of detailed output is shown below.

```
MM/DD/YY HH:MM      $PRO1   BAxxDDL     UPFR                PAGE:    1

Offset[Len]   Field Name and Description                     Data Type

RECORD UPFR



              The User/Product Files Relation File (UPFR) contains
              security information for each operator.  The purpose of
              the UPFR is to build the Virtual Menu screen, according
              to the files of a BASE24 product that each operator
              logging on has been allowed access.

              Each operator has one record for each BASE24 product
              file that exists in the BASE24 system.

              The following pages describe the fields included in a
              UPFR record.  The information below summarizes other
              information specific to the UPFR.

              LCONF ASSIGN:       UPFR


0     [404]

0     [18]    prikey

0     [16]    prikey.alias.byte[0:15]
                              [ALIAS]                     PIC X(16).


              The operator's alias, an identifier used to log on to
              the BASE24 CRT access system.
```

```
16    [2]     prikey.prod^id                    TYPE BINARY 16 SIGNED.

              Indicates the BASE24 product that the operator is
              permitted to access.  Values are defined in the
              PITABLE section of the PITABLE file.

18    [2]     num^files                       TYPE BINARY 16 UNSIGNED.

              Indicates the number of files the operator is permitted
              to access.

20    [384]   file^list[0:95]
                              [OCCURS DEPENDING ON NUM^FILES]

              The value in the following field identifies the files
              the operator is allowed to access.  The files indicated
              are specific to the BASE24 product identified by the

MM/DD/YY HH:MM      $PRO1   BAxxDDL     UPFR                PAGE:    2

Offset[Len]   Field Name and Description                     Data Type

              value in the PROD-ID field.

20    [4]     file^list[0:95].file^id.byte[0:3]             PIC X(4).

              A file, by acronym or abbreviation, that a user is
              permitted to access.

              END   [size  404]
```

Producing Detailed File Information

The DDLDOC utility can retrieve detailed file information from the DICTKDF and DICTRDF. The information returned includes the file name, record length, file type, file code, maximum extents, primary and secondary extents, and key information.

When detailed file information is requested, the DDLDOC utility includes the following fields after each record structure:

FILE NAME — The name of the file for which information is being provided.

RECLEN — The maximum record length for records in the file.

TYPE — The Guardian code identifying the type of file.

FILECODE — The Guardian file code for the file. If a file code is not assigned in the source schema file, the DDL compiler assigns a default file code of 0.

MAX EXTENTS — The maximum number of extents that can be allocated to the file. If the maximum extents are not defined in the source schema file, the DDL compiler assigns a default maximum of 100.

PRI EXTENT — The size of the primary extents for the file. If the primary extent size is not defined in the source schema file, the DDL compiler assigns a default primary extent size of 4.

SEC EXTENT — The size of the secondary extents for the file. If the secondary extent size is not defined in the source schema file, the DDL compiler assigns a default secondary extent size of 32.

KEYTAG — Indicates the number of the key for which the name, offset, and length are being provided. Key numbering starts with 0.

NAME — The name of the key identified by the KEYTAG field.

OFFSET — The offset of the key identified by the KEYTAG field from the beginning of the data structure.

LENGTH — The length of the key identified by the KEYTAG field.

Command Entry

To request detailed file information, include the FILE or FILE ON option on the command line, as shown in the following examples:

```
DDLDOC/OUT $S.#HOLD/FILE,COBOL,SHOW SAF
DDLDOC/OUT $S.#HOLD/FILE ON,SHOW REC SAF
```

Sample Detailed File Information

Sample output with detailed file information is shown below.

```
MM/DD/YY HH:MM      $PRO1   BAxxDDL     SAF                  PAGE:    1

POS      LVL  FIELD NAME AND DESCRIPTION                      DATA TYPE

RECORD SAF

1-4062    00  SAF
1-14      02  K1
1-2       04  DPC-ID                              TYPE BINARY 16 SIGNED.
3-8       04  TIMESTMP         [OCCURS    3]     TYPE BINARY 16 SIGNED.
9-10      04  UNIQUE-KEY                         TYPE BINARY 16 SIGNED.
9-10      04  MSG-LGTH                           TYPE BINARY 16 SIGNED.
                              [REDEFINES UNIQUE-KEY]
11-12     04  PROD-ID                            TYPE BINARY 16 SIGNED.
13-14     04  SEQ-NUM                            TYPE BINARY 16 SIGNED.
15-4062   02  SAVE-AREA                                    PIC X(4048).
              END   [size 4062]

FILE DATA from dictionary
    file name: SAF
       reclen:  4062
         type: K
     filecode:     0
  max extents:   100
   pri extent:     4
   sec extent:    32


   Keytag     0
     name K1
   offset     0
```

Creating and Using Input Files

Input files can be used as an alternate method of configuring and requesting DDLDOC output. They are especially useful for requesting more information than can be easily entered with a single run command, or for requesting a frequently printed series of information. For example, an input file that produces detailed output for all records in a data dictionary could be processed each time a change is made to the data dictionary, so that current information is available.

Input files can be created using any Compaq NonStop editor. The input file contains one DDLDOC option per line. The example on the following page shows an input file to request TAL format output without comments for all BASE24-atm files.

ACI provides a series of input files on the BA*xx*DDOC subvolume, where *xx* is the number of the current release, that can be used to produce DDLDOC output. These input files are described in appendix A.

Note: The input file in the example includes several lines of comments at the top of the file. The DDLDOC utility identifies comments in input files by a hyphen (-) in the first position on the line (excluding the question mark (?), if one is present).

The FRAT DDLDOC input file, which is used to generate the file and record DDLs for BASE24-atm, is shown below.

```
?-  ******************************************************
?-
?-  Files and Record Structures - BASE24-atm
?-
?-  ******************************************************
?-  *           Release 6.0                              *
?-  ******************************************************
?-  21MAY2001   jfu/543
?-  Symptom:    Release 6.0 Enhancements
?-  Problem:    None.
?-  Fix:        Added support for the atdd1 and atds1.
?-              Added support for the trf.
?-  Dependency: None.
?-  Reference:  WO #990108-1 (TDF Expansion)
?-              WO #981118-3 (Transactions Allowed)
?-  ******************************************************
?volume at60ddl

?show atdd1
?page
?show atdd1-core
?page
?show atds1
?page
?show atds1-core
?page
?show rcpt
?page
?show rpt3mstr
?page
```

```
?show rpt3ttl
?page
?show rpt5mstr
?page
?show rpt5ttl
?page
?show rpt
?page
?show tdf
?page
?show tlf
?page
?show tlf-setl-ttl
?page
?show tlf-term-cash
?page
?show tlf-term-setl
?page
?show trf
```

Section 3

File, Table, and Record Structure Locations

For the most part, BASE24 file and record structures are located in compiled data dictionaries on DDL subvolumes. Shared file structures are located in the data dictionary on the Base DDL subvolume, named BA*xx*DDL, where *xx* is the number of the current release. Product-specific file structures are located in data dictionaries on product-specific subvolumes; for example, BASE24-atm files are on the AT*xx*DDL subvolume, and BASE24-pos files are on PS*xx*DDL or BP*xx*DDL subvolumes.

SQL table structures are located in either DDL source schema files or SQLCI input files located on product-specific subvolumes.

This section identifies the location of BASE24 file, table, and record structures information.

Note: Throughout this section, the *xx* characters in subvolume names always represent the number of the current release.

BASE24 Standard Product File and Table Locations

The following table identifies the standard BASE24 files and tables. For each BASE24 file or table, this table provides the following information:

- The acronym associated with the file or table
- The file or table name
- The product that uses the file or table
- The standard location of the data dictionary, DDL source schema file, or SQLCI input file from which information about the file or table can be retrieved

Note: Device-specific files and tables as well as device-specific data structures stored in shared files or tables are not identified in this table. The data dictionary, DDL source schema file, or SQLCI input file from which information about the file or table can be retrieved, are always located on the device-specific subvolume.

For the most part, the DDL record name is the same as the file acronym. For data dictionary locations, the subvolume name is provided. BASE24-billpay subvolumes include a three-character internal release number (e.g., A04) in the subvolume name, which is denoted by the variable *xxx* in the following table. For DDL source schema files and SQLCI input file locations, a subvolume and file names are provided. The information in SQLCI input files and DDL source schema files that are not compiled in a data dictionary cannot be viewed using the DDLDOC utility.

Acronym	File or Table Name	Product	Location
ADMN	Administrative Card File	BASE24-pos	BP*xx*DDL
ADRT	Address Table File	Core	OC*xx*DICT
ANEG	American Express National Negative Card File	Shared file	BP*xx*DDL
AORF	Assign/Object Relationship File	Kernel	OK*xx*DICT
APCF	Acquirer Processing Code File	Shared file	BA*xx*DDL

Acronym	File or Table Name	Product	Location
APRF	Assign/Process Type Relationship File	Kernel	OK*xx*DICT
AQF	Administrative Queue File	BASE24-pos	BP*xx*ADMN
ARF	Account Routing File	Shared file	BA*xx*DDL
ASGN	Assign File	Kernel	OK*xx*DICT
AST	Authorization Selection Table	BASE24-pos	BP*xx*DDL
ATDD1	Terminal Data Dynamic File—general data	BASE24-atm	AT*xx*DDL
ATDD2	Terminal Data Dynamic File—scratch pad	BASE24-atm	AT*xx*DDL
ATDS1	Terminal Data Static File—general data	BASE24-atm	AT*xx*DDL
ATT	Account Type Table File	Shared file	BA*xx*DDL.ATTDS
BANK	Bank Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
BBLS	Bank Billing Summary Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
BKAV	Bank Transaction Advice Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
BKBR	Bank Branch Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
BKNT	Bank Notice Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
BKTS	Bank Transaction Summary Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
BLG	Billing Group Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
BLRT	Billing Rate Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS

Acronym	File or Table Name	Product	Location
BLTY	Billing Type Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
BNEG	Base National Negative Card File	Shared file	BP*xx*DDL
BNR	Base National Range File	Shared file	BP*xx*DDL
BSF	Bin Sort File	BASE24-atm	AT*xx*CDDL
CACT	Customer/Account Relation Table	Core	OC*xx*CUST.CACTRS
CAF	Cardholder Authorization File	Shared file	BA*xx*DDL
CAPF	Card Authorization Parameters File	BASE24-pos	BP*xx*DDL
CART	Check Authorization Routing Table File	BASE24-pos	PS*xx*DDL
CATT	Customer Allowed Transaction Table	Core	OC*xx*CUST.CATTRS
CBLS	Customer Billing Summary Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
CBPT	Customer Billpay Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
CCF	Corporate Check File	BASE24-atm	AT*xx*CDDL
CCIF	Customer/Cardholder Information File	BASE24 Customer Service	OC*xx*DICT
CCMF	Customer/Card Memo File	BASE24 Customer Service	OC*xx*DICT
CHF	Card History File	BASE24-pos	BP*xx*CRTA
CLPT	Cleanup Configuration Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
CNTX	Context Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS

Acronym	File or Table Name	Product	Location
CPF	Card Prefix File	Shared file	BA*xx*DDL
CPIT	Customer/Personal ID Relation Table	Core	OC*xx*PSNL.CPITRS
CRUF	CRT Authorization Retailer Usage File	BASE24-pos	BP*xx*CRTA
CSEC	CRT Authorization Security File	BASE24-pos	BP*xx*CRTA
CSF	Check Status File	BASE24-atm	AT*xx*CDDL
CSTT	Customer Table	Core	OC*xx*CUST.CSTTRS
CVND	Customer Vendor Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
CVS	Customer Vendor Summary Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
DEST	Destination File	Integrated Endpoint Process	OE*xx*DICT
ECF	Extract Configuration File	Shared file	BA*xx*DDL
ELF	Exception Log File	Core	OC*xx*DICT
EMF	External Message File	Shared file	BA*xx*DDL
EMT	Extended Memory Table File	Kernel	OK*xx*DICT
EORF	EMT/Object Relationship File	Kernel	OK*xx*DICT
EPRF	EMT/Process Type Relationship File	Kernel	OK*xx*DICT
ERF	Exchange Rate File	Shared file	BA*xx*DDL
ESF	Event Suppression File	Kernel	OK*xx*DICT
FCF	File Control File	Core	OC*xx*DICT
FUTR	Future Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS

Acronym	File or Table Name	Product	Location
HCF	Host Configuration File	Shared file	BA*xx*DDL
HIST	Billpay History Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
HMBF	Host Mail Box File	BASE24-mail	MA*xx*DDL
HMTF	Host Message Text File	BASE24-mail	MA*xx*DDL
ICF	Interchange Configuration File	Shared file	BA*xx*DDL
ICFE	Enhanced Interchange Configuration File	Shared file	BA*xx*DDL
IDF	Institution Definition File	Shared file	BA*xx*DDL
IFCF	ITS Interface Configuration File	Core	OC*xx*DICT
ILF	Interchange Log File	Shared file	BA*xx*DDL
IMTF	Internal Message Translation Configuration File	Core	OC*xx*DICT
INEG	INAS National Negative Card File	Shared file	BP*xx*DDL
INR	INAS National Range File	Shared file	BP*xx*DDL
IPCF	Issuer Processing Code File	Shared file	BA*xx*DDL
IPRF	Institution Periodic Reporting File	Core	OC*xx*DICT
IRCF	Institution Routing Configuration File	Core	OC*xx*DICT
ISF	Interface Security File	Core	OC*xx*DICT
IST	Interface Station Table File	Core	OC*xx*DICT
ITLF	ITS Transaction Log File	Core	OC*xx*DICT
KEYA	Key Authorization File	Shared file	BA*xx*DDL
KEYD	Derivation Key File	Shared file	BA*xx*DDL

Acronym	File or Table Name	Product	Location
KEYF	Key File	Shared file	BA*xx*DDL
KEYI	ICC Key File	Shared file	BA*xx*DDL
KEY6	Key 6 File	Shared file	BA*xx*DDL
LCF	Log Configuration File	Core	OC*xx*DICT
LCONF	Logical Network Configuration File	Shared file	BA*xx*DDL
LOAN	Loan Application Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
LPF	Log Permission Process Partition File	Kernel	OK*xx*DICT
MBAF	Mail Box Address File	BASE24-mail	MA*xx*DDL
MBF	Mail Box File	BASE24-mail	MA*xx*DDL
MISC	Miscellaneous Request Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
MTF	Message Text File	BASE24-mail	MA*xx*DDL
NBF	No Book File	BASE24-teller	TR*xx*DDL
NEG	Negative Card File	Shared file	BA*xx*DDL
OBJ	Object File	Kernel	OK*xx*DICT
OMF	Online Maintenance File	Shared file	BA*xx*DDL
ORF	Override Response File	BASE24-teller	TR*xx*DDL
PBF	Positive Balance File	Shared file	BA*xx*DDL
PCDF	Processing Code Definition File	Core	OC*xx*DICT
PDF	Processing Code Description File	Shared file	BA*xx*DDL
PIRF	POS Institution Reporting File	BASE24-pos	PS*xx*DDL

Acronym	File or Table Name	Product	Location
PIT	Personal Information Table	Core	OC*xx*PSNL.PITRS
PORF	Process/Object Relationship File	Kernel	OK*xx*DICT
PRDF	POS Retailer Definition File	BASE24-pos	PS*xx*DDL
PRF	POS Referral File	BASE24-pos	BP*xx*DDL
PRO	Process File	Kernel	OK*xx*DICT
PRRF	POS Retailer Reporting File	BASE24-pos	PS*xx*DDL
PTDD1	BASE24-pos Terminal Data Dynamic File—general data	BASE24-pos	PS*xx*DDL
PTDD2	BASE24-pos Terminal Data Dynamic File—scratch pad	BASE24-pos	PS*xx*DDL
PTDS1	BASE24-pos Terminal Data Static File—general data	BASE24-pos	PS*xx*DDL
PTDF	POS Terminal Data File	BASE24-pos	PS*xx*DDL
PTF	Process Type File	Kernel	OK*xx*DICT
PTLF	POS Transaction Log File	BASE24-pos	PS*xx*DDL
RATE	Rate Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
RATG	Rate Group Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
RCPT	Receipt File	BASE24-atm	AT*xx*DDL
RCUR	Recurring Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
RGCF	Refresh Group Control File	Shared file	BA*xx*DDL
RPT	Report Files	BASE24-atm	AT*xx*DDL
RPTA	Report Action Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS

Acronym	File or Table Name	Product	Location
RTBL	Routing Table File	BASE24-pos	BP*xx*DDL
RTRY	Retry Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
RTTB	Report by Time Block File	BASE24-teller	TR*xx*DDL
RTTF	Report by Type File	BASE24-teller	TR*xx*DDL
RTXT	Response Text Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
SAF	Store-and-Forward File	Shared file	BA*xx*DDL
SCM	Source Class Map File	Kernel	OK*xx*DICT
SCT	Server Class Type File	Kernel	OK*xx*DICT
SEC	Security File	Shared file	BA*xx*DDL
SPF	Stop Payment File	Shared file	BA*xx*DDL
SPRF	Server Class Type/Process Relationship File	Kernel	OK*xx*DICT
STF	Switch Terminal ID File	Shared file	BA*xx*DDL
SURF	Surcharge File	Shared file	BA*xx*DDL
SVHF	Stored Value History File	BASE24-pos	PS*xx*DDL
SVPT	Service Provider Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
TCF	Transaction Code File	Shared file	BA*xx*DDL
TDF	Terminal Data File	BASE24-atm	AT*xx*DDL
THCF	Transaction History Configuration File	Core	OC*xx*DICT
THF	Transaction History File	Core	OC*xx*DICT
TKN	Token File	Shared file	BA*xx*DDL
TLF	Transaction Log File	BASE24-atm	AT*xx*DDL

Acronym	File or Table Name	Product	Location
TRF	Terminal Receipt File	BASE24-atm	AT*xx*DDL
TTDF	Teller Terminal Data File	BASE24-teller	TR*xx*DDL
TTF	Teller Transaction File	BASE24-teller	TR*xx*DDL
TTLF	Teller Transaction Log File	BASE24-teller	TR*xx*DDL
TTSC	Transaction Type By Source Code Table	Core	OC*xx*DDL.TTSCRS
TQF	Terminal Queue File	BASE24-pos	BP*xx*CRTA
UAF	Usage Accumulation File	Shared file	BA*xx*DDL
UFIR	User/FIID Relations File	Shared file	BA*xx*DDL
ULF	Update Log File	BASE24-from host maintenance	FH*xx*DDL
UPFR	User/Product Files Relations File	Shared file	BA*xx*DDL
VBUD	Vendor Budget Category Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
VNDB	Vendor Branch Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
VNDM	Vendor Mask Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
VNDR	Vendor Table	BASE24-billpay	BPPM*xxx*S.DDLSQLS BPPM*xxx*S.DDLDEFS
WHFF	Warning/Hold/Float File	BASE24-teller	TR*xx*DDL

Device-Specific File Locations

Some devices require device-specific files. When a file is associated with a particular device, the file structure information for that file is located on the source code subvolume for the device's Device Handler process. For example, the Operator Identification File (OIF) contains operator access information for Diebold 10XX/478X terminals. The file structure information for the OIF is on the AT*xx*1000 subvolume, along with the Diebold 10XX/478X Device Handler source code.

In addition, some files contain device-specific information. When device-specific information is associated with a particular file, the device-specific information structure is located on the source code subvolume of the device's Device Handler process. For example, the device-specific static information for Diebold 10XX/478X terminals is located on the AT*xx*1000 subvolume.

Interchange-Specific File Locations

Interchange-specific file structures are located on the source code subvolume for the interchange. For example, the PLUS ISO Interchange Interface stores Proprietary Member Center IDs in the PLUS Proprietary Member Center File (PPF). PPF file structure information is located on the SW*xx*PISO subvolume.

Refresh Record Locations

The format of the refresh tape records does not match, field for field, the format of the disk file records being refreshed. Some fields in each disk file record cannot be refreshed. Therefore, the refresh tape or disk records contain only selected fields that can be refreshed.

Refresh record structures are located on the same subvolume as the corresponding file's record structure. These subvolumes are identified in the ***BASE24 Refresh and Extract Operators Manual***. Refresh tape headers and trailers are also documented in this manual.

In addition, the printed DDLDOC output for refresh records is provided on the BASE24 Product Documentation CD-ROM.

Extract Record Locations

The Super Extract process normally extracts records in a binary/ASCII format that matches the standard disk record format. The Super Extract process also supports two types of record conversions: ASCII-to-EBCDIC and binary-to-ASCII. Each is described below.

ASCII-to-EBCDIC Conversion

The Super Extract process can convert values in ASCII fields in extract records to EBCDIC. When this option is used, the Super Extract process converts the values in all ASCII fields to EBCDIC while leaving the binary fields untouched. Tape labels are always in EBCDIC. Converting the values in ASCII fields to EBCDIC does not affect the field lengths documented in the DDLs.

Binary-to-ASCII Conversion

The Super Extract process can convert the values in all binary fields on the tape to ASCII character format. If this option is used, values in all binary fields are converted to ASCII. This includes tape and file headers and trailers, extract records, tokens, record headers, and block headers. The only data on the tape not included in the conversion are the IBM volume or other IBM labels. The ASCII-only format is only available for the files listed below.

- Interchange Log File (ILF)
- ITS Transaction Log File (ITLF)
- POS Transaction Log File (PTLF)
- Store-and-Forward File (SAF)
- Transaction Log File (TLF)
- Teller Transaction Log File (TTLF)

The table below identifies the standard subvolume and DDL definition name of the extract format for each of these files.

File Acronym	Subvolume	Definition Name
ILF	BA*xx*DDL	ILFX
ITLF	OC*xx*DICT	ITS-TLFXA (ASCII format) ITS-TLFXB (Binary format)
PTLF	PS*xx*DDL	PTLFX-CLERK-TOT PTLFX-SET-REC PTLFX-SRVCS-1 PTLFX-SRVCS-2 PTLFX
SAF	BA*xx*DDL	SAFX
TLF	AT*xx*DDL	TLFX-SETL-TTL TLFX-TERM-CASH TLFX-TERM-SETL TLFX
TTLF	TR*xx*DDL	TTLFX

In addition, the printed DDLDOC output for extract records is provided on the BASE24 Product Documentation CD-ROM.

Tokens in ASCII-Only Extracts

Token data in extract records can be in either binary or ASCII format. The binary and ASCII formats for a token contain the same fields. The only difference is that binary fields are converted to numeric fields in the ASCII format.

When the ILF, ITLF, PTLF, TLF, or TTLF is being extracted in ASCII-only format, the ASCII format of the token is used. Token structures are documented in the ***BASE24 Tokens Manual***.

Extract Tape Headers and Trailers

Extract tape headers and trailers are documented in the ***BASE24 Refresh and Extract Operators Manual***.

Lexicon Structure Locations

Lexicons are logical groupings of all the member functions that a client object can request of an actor object. Member functions are used by BASE24 kernel, integrated endpoint, and core objects to communicate with one another. Each member function includes a request and response message structure, which are defined to the data dictionary using the DEFINITION statement. Currently, only the Process Control object lexicons and the lexicon header can be displayed using the DDLDOC utility. These lexicon structure definitions are located on the OK*xx*DICT subvolume.

ACI Worldwide Inc.

Appendix A

ACI-Supplied DDLDOC Input Files

The BA*xx*DDOC subvolume contains a number of input files for use with the DDLDOC utility. This appendix describes the naming convention for the input files and provides some tips on using the input files.

Input File Names

The files on the BA*xx*DDOC subvolume are organized by function, product, and module or add-on product. The naming convention for these input files is *ffppxxxx*, where *ff* identifies the function, *pp* identifies the product abbreviation, and *xxxx* identifies the add-on product or module.

Valid functions include the following:

EB = Binary format extract records
ED = Display format extract records
FR = File and record structures
LX = Lexicon message structures
RD = Display format refresh records
TB = Binary format token structures
TD = Display format token structures

Product abbreviations include the following:

AT = BASE24-atm
BA = Base (shared files)
BP = Base POS
FH = BASE24-from host maintenance
MA = BASE24-mail
OC = Core
OE = Integrated endpoint
OK = Kernel
PS = BASE24-pos
TR = BASE24-teller

Modules and add-on product identifiers include the following:

ADMN = BASE24-pos CRT Administration structures
CRTA = BASE24-pos CRT Authorization structures
HPDH = BASE24-pos Hypercom Device Handler structures
MHI = BASE24-pos Merchant Host Interface structures
NCD = Non–Currency Dispense
SM = Smart Card
SPDH = BASE24-pos Standard POS Device Handler structures
SSBB = BASE24-atm Self-Service Banking (SSB) Base application structures
SSBC = BASE24-atm Self-Service Banking (SSB) Check application structures

For example, the input file FRBA contains all the necessary DDLDOC command options to create the file and record structure information for the Base files. The input file RDTR contains the DDLDOC command options to print refresh display formats for BASE24-teller.

Using the Input Files

The input files direct the DDLDOC utility to use the default format options. By adding other format options to the command line, however, you can use the input files to produce other formats. For example, if you enter the command:

```
DDLDOC/IN FRATSSBB, OUT $S.#HOLD/
```

The output produced is in TAL format, without detailed field descriptions. By adding the options ?COBOL and ?DETAIL to the command line, detailed, COBOL format output is produced. The revised run command is as follows:

```
DDLDOC/IN FRATSSBB, OUT $S.#HOLD/?COBOL,?DETAIL
```

You can modify these input files to contain any combination of DDLDOC commands you want.

Note: The command line can be used to override any default format (that is, any format option which is not explicitly specified in the input file). However, if a format is explicitly named in the input file, it cannot be overridden by an entry on the command line. For example, if your input file produces TAL format output because no other format option (?COBOL, ?NOTAL, or ?TAL) is specified in the input file, you can request COBOL format output from the command line. However, if your file produces TAL format output because the ?TAL option is included in the input file, entering the ?COBOL option on the command line does not change the format. For this reason, you may want to consider not placing format options in the input files. If the input files do not contain format options, the input file can easily be used for any format by simply identifying the format option on the command line.

Index

Special Characters

C

D

E

F

H

I

L

M

N

O

P

R

S

T

U

V

ACI Worldwide Inc.